

Predictable and Secure System Auditing for Real-Time Systems

Peng Jiang¹, Fanhang Hu¹, Ruizhe Huang², Shuomin Xue¹, Zhaomeng Deng², Yuxin Ren³,
Ning Jia³, Yao Guo², Xiangqun Chen², Ding Li², Guang Cheng¹

¹Southeast University, China

²Key Laboratory of High-Confidence Software Technologies, School of Computer Science, Peking University, China

³Huawei Technologies Co., Ltd, China

Abstract—System auditing frameworks are essential for operating system security as they record system events to support intrusion detection, compliance verification and attack reconstruction. However, existing auditing frameworks fail to meet the stringent requirements of real-time systems, which demand security, predictability, and efficiency. Though current solutions are optimized for security or performance, they do not focus on bounding the worst-case execution time (WCET) and incorporating into response-time analysis (RTA).

This paper presents **RT-NODROP**, a secure and predictable auditing framework tailored for real-time systems. **RT-NODROP** employs a lightweight threadlet-based architecture to isolate audit events processing, periodically invoking threadlets to simultaneously bound WCET and event residence time. By integrating with real-time schedule, **RT-NODROP** ensures no event dropping, system efficiency, and predictability. We further develop an overhead-aware RTA and a period selection algorithm to balance security, performance, and schedulability. The evaluations demonstrate that **RT-NODROP** is superior over state-of-the-art frameworks (Sysdig, OMNILOG, Ellipsis), improving schedulability by 80.11%, 117.9% and 51.05%, respectively. For the latency-intensive application Redis, **RT-NODROP** achieves up to 75.1% (138.86%, 324.6%) higher throughput and 2.19× (3.07×, 5.02×) lower 99.9th percentile tail latency than Sysdig (OMNILOG, Ellipsis) while maintaining a minimum event residence time around 10ms without event dropping.

I. INTRODUCTION

Auditing frameworks [1], [2], [3], [4], [5], [6], [7], [8], [9] are critical operating system security infrastructure for intrusion detection, provenance analysis, compliance verification, and attack reconstruction [10], [11], [12], [13], [14], [15], [16], [17]. System auditing has achieved widespread deployment across a spectrum of computational environments, ranging from host-level, to cluster-wide, and to enterprise-scale infrastructures [18], [19], [20]. Especially, since security-critical real-time and latency-sensitive systems face increasing vulnerabilities and attacks [21], [22], [23], [24], more and more real-time systems are incorporating and leveraging auditing frameworks [8], [25]. For instance, we observe the deployment of auditing frameworks in multiple security- and time-critical productions, such as online transaction processing systems that support mobile payment and telecom core network management systems that support the 5G ultra-reliable low-latency communication pipelines [26], [27], [28].

A typical auditing framework collects system call (syscall) events, analyzes events to detect and investigate attacks, and

persistently stores events. An auditing framework is comprised of two components. A *kernel module* is responsible for monitoring and tracing syscalls, and sends events to a *user-space process* for further processing and analysis. Unsurprisingly, attackers are actively engaged in compromising auditing frameworks, given their criticality. Recent studies have highlighted that existing auditing frameworks are vulnerable to two threats [4], [5], [25], [29]. 1) Adversaries exploit the prolonged residence time of audit events within the kernel by modifying or deleting these events before they can be submitted for processing. 2) An adversary generates a large number of events at a rate that exceeds the framework processing speed, overwhelming the auditing framework to drop events related to malicious actions.

However, implementing a secure auditing framework on real-time systems poses significant challenges, as it must balance security, predictability, and efficiency. Real-time system auditing must meet the following requirements. 1) All events must be collected and processed without loss. 2) The events' duration in the insecure kernel buffer must be strictly bounded. 3) It is essential to have predictable execution to guarantee schedulability. 4) Minimizing the auditing overhead is necessary to prevent considerable system performance degradation.

Unfortunately, current auditing frameworks optimize solely for either security or performance, thereby preventing their usage in real-time systems. The fundamental issue is that their design does not respect the intrinsic temporal constraints and requirements of real-time systems. ***They do not optimize the worst-case execution time (WCET), leading to unpredictable and unbounded interference, and their execution model cannot be adapted to real-time schedules and incorporated into response time analysis (RTA) frameworks.*** For example, NODROP [5] leverages a threadlet-based [30] approach to achieve a synchronized, higher priority audit event processing, thereby ensuring lossless processing of all events. However, NODROP accumulates a huge number of events for batch processing, resulting in an extremely large WCET of event processing and unbounded event residence time. OMNILOG [6] guarantees near-zero event residence time by immediately transmitting each audit event to a hypervisor-enforced isolated buffer, but imposes an expensive hypercall to each syscall execution, significantly increasing the WCET of real-time applications. The state-of-the-art real-time system auditing

framework is Ellipsis [8], which utilizes the periodic repetitive nature of applications to reduce the audit event volume. While Ellipsis minimizes interference to real-time applications, it inherits the Linux Auditd architecture, suffering from event loss and tampering threats from malicious attackers.

This paper presents RT-NODROP, an auditing framework tailored for real-time systems that simultaneously ensures security, efficiency, and predictability. RT-NODROP adopts a threadlet-based architecture to segregate the processing of audit events originating from distinct threads into dedicated threadlets. The key idea in RT-NODROP is that periodically invoking threadlets and individually determining their periods based on application syscall frequency effectively integrates the auditing framework within real-time schedules. Since invoking and switching lighter-weight threadlets in RT-NODROP incurs much less system overhead than current auditing frameworks, improving the overall system efficiency. Furthermore, periodic audit event processing is essential to simultaneously bound the WCET of event processing, the event residence time, and a buffer size large enough to avoid event dropping. Consequently, RT-NODROP effectively avoids security threats in existing auditing frameworks.

We determine the timing properties of auditing frameworks and conduct an auditing overhead-aware RTA. This analysis jointly calculates bounds for event processing interference and event residence time. Moreover, to achieve system schedulability and desired event residence time, we introduce an auditing period selection algorithm. This algorithm realizes a controlled trade-off among system overhead, schedulability, and security risks. This algorithm further enables system administrators to flexibly customize constraints and objectives for event residence time according to their specific production demand.

RT-NODROP is implemented as a variant of NODROP. We perform schedulability tests to evaluate the impact of RT-NODROP on system schedulability compared to current auditing frameworks. In the multi-core configuration, RT-NODROP improves the average schedulability by 80.11%, 117.9% and 51.05% over Sysdig, OMNILOG and Ellipsis, respectively. We also measure the improvement of Schedulable Utilization Areas, and find that RT-NODROP outperforms Sysdig, OMNILOG and Ellipsis by 11.63%, 21.98% and 8.64%, respectively. We apply RT-NODROP to Reids, a popular key-value store used in latency-sensitive environments. Our experiments show that RT-NODROP achieves up to 75.1%, 138.86% and 324.6% higher throughput and 2.19, 3.07 and 5.02 times lower 99.9th percentile tail latency than Sysdig, OMNILOG and Ellipsis, respectively. Especially, RT-NODROP maintains a minimum event residence time around 10ms without event dropping, confirming its robustness in ensuring security and predictability.

This paper centers on providing WCET-focused auditing framework in a real-time and latency-sensitive environment. The contributions include:

- We propose a real-time system auditing framework, that has predictable execution, guaranteed security, and high performance.

- We conduct an auditing overhead aware RTA and apply it to analyze multiple auditing frameworks.
- We present a auditing event processing period selection algorithm that optimizes the event residence time while ensuring system schedulability.

II. BACKGROUND

A. System Auditing Background and Terms

Originating from the fundamental need for operating system security monitoring, system auditing enables continuous monitoring and recording of security-related events, providing essential data support for post-incident tracing, attack detection, and compliance verification. A typical system auditing framework [2], [9] operates under a producer-consumer model and employs an architecture consisting of three key components. The first component is a kernel-space event collector that captures system activities (e.g., syscalls of applications) through hooks in the kernel, produces the corresponding audit events, and saves events in an insecure kernel ring buffer temporarily. The second component is a kernel-user space audit event transmission mechanism that uses message queues or zero-copy methods to send audit events from the kernel buffer to the event processing component, which is the third key component of auditing frameworks. We call the event processing component a consumer, which usually is a user-space daemon process that performs event processing, filtering, and storage. The event residence time is defined as the duration spanning from the moment an event is collected by the kernel to the point at which the consumer completes the processing of that event.

B. Security Threats in System Auditing

Attackers have noticed the value of audit events and try to compromise system auditing to conceal their malicious activities. There are two common attacks.

Race Condition Attack. Paccagnella et al. [29] proposed an attack that deletes audit events generated by malicious processes from the kernel buffers before they have been submitted to the consumer. The key insight of this attack is that an audit event may stay in the kernel buffer up to 250 ms before the consumer can process it. Therefore, adversaries have enough time to manipulate audit events in the kernel buffer. For example, loading and executing a malicious kernel module by `insmod` only takes tens of milliseconds [4], [6].

Flooding Attack. The flooding attack consists of two steps. First, the adversary runs a program that forks multiple subprocesses. All subprocesses simultaneously generate a large number of events. Second, the adversary starts malicious actions along with the flood generated in the previous step. When the consumer is saturated, the kernel buffer will be *fully occupied* by the events and stop recording the malicious actions [5], [25].

To ensure the security and high availability, system auditing should prevent event dropping even under high system workloads and minimize event residence time in kernel space.

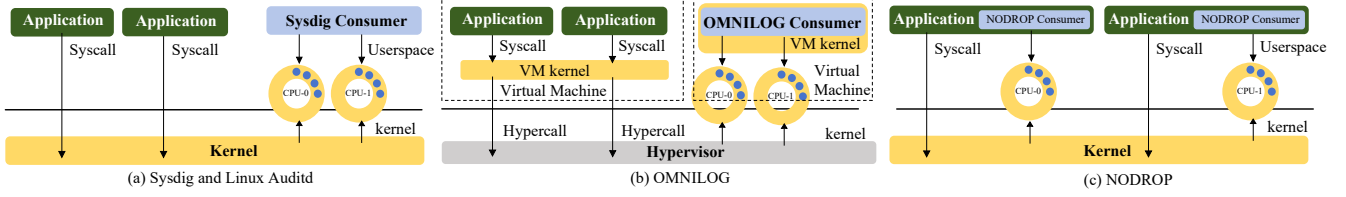


Fig. 1: Architectures of different system auditing frameworks.

C. Existing Auditing Frameworks

This section introduces the architecture of some representative auditing frameworks.

As shown in Fig. 1 (a), current typical auditing frameworks, such as Sysdig [9], Linux Auditd [2], and Ellipsis [8], follow an asynchronous architecture. This architecture intercepts tasks' syscalls and asynchronously processes audit events by a single-threaded user-space consumer. To make the consumer more efficient, Sysdig employs a buffer-aware adaptive sleeping mechanism, where the consumer actively polls the buffer for new events; if the buffer is empty, it briefly sleeps (millisecond-scale) to reduce CPU usage, waking promptly when new events arrive. Ellipsis inherits the Linux Audit architecture and leverages the predictable nature of RTOS to perform aggressive, kernel-level event reduction through template matching. However, their asynchronous architecture leads to both performance and security issues under heavy system workloads. The consumer accesses events generated from all CPU cores, resulting in considerable cross-core memory copies and cache coherence overhead that slows down both the consumer and applications. Events are temporarily stored in an insecure kernel buffer, where they remain for a long while until the consumer wakes up to read them, risking both a race condition attack and a flooding attack. To prevent event dropping, some Sysdig variant blocks all applications executing syscalls when the buffer nears capacity, significantly degrading the whole system performance [5].

OMNILOG [6] adopts a synchronous architecture to protect audit events as shown in Fig. 1 (b). In the syscall execution path, it constructs and transmits the event to a trusted component before the syscall is allowed to proceed. Specifically, the monitored host operates as a virtual machine, the trusted component is embodied by the hypervisor, while the event transmission is realized by a self-defined hypercall. The consumer runs in a separate virtual machine and focuses on persisting these protected events from the hypervisor buffer to a secure storage device. Though the event residence time can be near-zero, this synchronous transmission method can block applications, leading to significant syscall delays due to hypercall and virtualization overhead.

As shown in Fig. 1 (c), NODROP [5] offers a more scalable solution with per-thread consumer architecture where each consumer resides in the memory space of its host thread with some form of memory protection [31], [32], [33]. The consumer is implemented using threadlet [30], a short se-

quence of instructions with self-contained memory. The kernel module stores audit events in a per-thread logging buffer. When the buffer is full or the monitored host thread exits, control is passed to the consumer, which preempts the host thread and processes events using the host thread's resource quota. Thus, the consumer in NODROP implicitly has higher priority than the host thread, but has lower priority than other higher priority threads. The buffer is shared with the consumer to transfer events without copying. After processing, control is returned to the host thread. This alternating execution between the host thread and the consumer ensures that each thread handles its own events without affecting others, while also minimizing overhead by leveraging the native operating system process isolation mechanisms and reducing the context switches between threads. It also prevents event dropping. However, NODROP's current mechanism of triggering the consumer when the buffer is full does not guarantee the timeliness of event processing because NODROP uses a large buffer to amortize consumer execution cost. For threads that generate syscalls slowly, the buffer may remain unfilled for extended periods, causing the consumer to defer processing and resulting in a long event residence time.

III. RESEARCH CHALLENGES

A. Research Goals

Designing an auditing framework for real-time systems requires balancing three objectives among security, predictability, and efficiency.

Event integrity. All audit events must be collected and processed on time without loss.

Bounded event residence time. The duration of events that reside in the insecure kernel buffer should be bounded. Furthermore, administrators can flexibly control the duration depending on task criticality or sensitivity.

Guaranteed schedulability. The execution and interference of the auditing framework should be bounded and can be adapted to response time analysis to evaluate system schedulability.

Minimized overhead. The auditing framework should not occupy too many system resources that significantly slow down the whole system even under high-frequency event generation in multi-core environments.

B. Predictability Issues in Existing Frameworks

Existing frameworks fail to meet these goals in §III-A due to inherent architectural defects. These frameworks typically

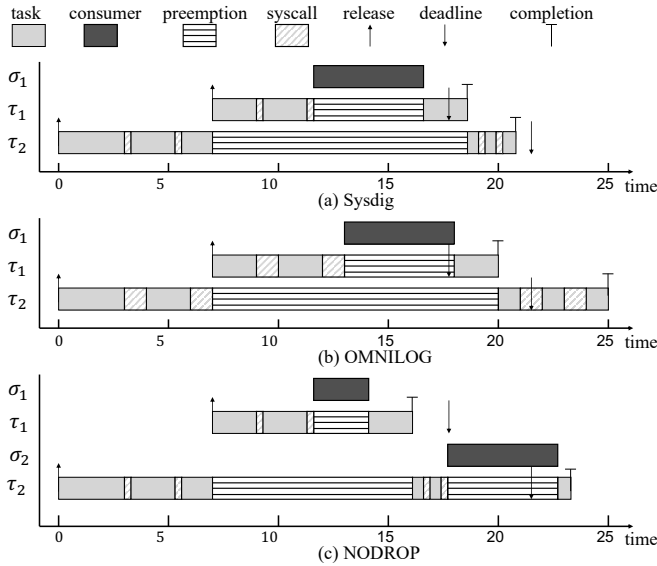


Fig. 2: Example schedules of two tasks and the corresponding consumers. τ_1 has higher priority, is released at 7, its WCET is 6, its period is 12, and its deadline is 19. τ_2 has lower priority, is released at 0, its WCET is 8, its period is 21, and its deadline is 21. (a) Sysdig consumer process events from all tasks and incurs priority inversion, causing τ_1 to miss its deadline. (b) OMNILOG introduces significant syscall execution cost, further leading to deadline miss in τ_2 . (c) While NODROP avoids deadline miss in τ_1 thanks to per-thread consumer, its batch processing increases consumer WCET and causes τ_2 to miss deadline.

optimize one objective while neglecting the predictability. Consequently, they suffer from aggravated worst-case overhead and are prone to issues such as unbounded interference or priority inversion, which significantly degrade system schedulability. Moreover, their execution models cannot effectively be adapted to response time analysis, preventing efficient and reliable deployment in real-time systems.

Sysdig. In addition to performance overhead and event dropping issues discussed in §II-C, the single consumer design in Sysdig introduces a dilemma in its predictability. The challenge is how to determine the priority of the consumer. Low priority deteriorates the security risk of event dropping and residence time. Hence, the industry usually deploys high-priority consumers in practice. However, high priority also has two issues with its predictability. 1) *Priority inversion.* As shown in Fig. 2(a), events generated by low-priority tasks will inadvertently interfere with high-priority tasks via the consumer execution. Even worse, in case of multi-core environments, Sysdig will incur cross-core interference [34] that one task in one core can delay another unrelated task in a different via the consumer. 2) *Unbounded event residence time.* Sysdig consumer will sleep when the system load is low. While this saves resource usage, it leads to extended events residing in the kernel buffer.

OMNILOG. Similar to Sysdig, OMNILOG also leverages a single consumer that incurs the same priority inversion issue. OMNILOG optimizes event residence time by combin-

ing synchronous event transmission and hypervisor-enforced isolation. Hence, OMNILOG achieves near-zero event residence time. However, the immediate event transmission and virtualization introduce substantial execution overhead in OMNILOG. 1) *Syscall overhead.* As illustrated in Fig. 2(b), every syscall in OMNILOG invokes a hypercall and multiple memory copies to transfer the event to the secure hypervisor buffer, resulting in a slowdown of each syscall. 2) *Virtualization overhead.* OMNILOG mandates applications run inside virtual machines, but it is well known that virtual machines introduce substantial performance overhead compared to running applications directly on bare metal. This overhead arises from the additional layers of page tables, expensive hypervisor involvement, and slower device access [35], [36], [37], [38].

NODROP. By utilizing a per-thread consumer architecture, NODROP avoids priority inversion as the consumer is dedicated to handling events exclusively for its monitored host thread. Additionally, invoking NODROP consumer is more efficient than other frameworks, which need costly context switches between processes. Moreover, NODROP ensures no event dropping. However, NODROP is not designed for real-time systems and exhibits three predictability issues depicted in Fig. 2(c). 1) *Increased consumer interference.* Using multiple consumers in NODROP increases the consumer invocation frequency and associated system overhead. A low-priority task may be preempted and interfered with by all consumers corresponding to higher-priority tasks. NODROP lacks a fine-grained mechanism to control consumer invocations, thereby impeding the ability to analyze and quantify the extent of consumer interference. 2) *Long event residence time.* To amortize consumer invocation cost, NODROP leverages a huge kernel buffer and invokes the consumer only when the buffer is full. Even worse, all applications in NODROP uses the same buffer size regardless of their load. Therefore, applications with low syscall frequencies experience prolonged event residence time. 3) *Burst event processing.* The huge buffer introduces another issue that the consumer has to process a substantial volume of events accumulated in the buffer at one invocation. This burst processing significantly increases the worst-case consumer execution time. Thus, even if consumer invocations are infrequent, the worst-case interference caused by NODROP consumer can result in applications missing their deadlines.

Ellipsis. Ellipsis fundamentally improves predictability compared to Linux Audit. By reducing repeated log events and lowest-priority consumer, Ellipsis ensures to be schedulable within real-time constraints. However, it introduces two extra issues. 1) The lowest-priority Ellipsis consumer minimizes inference to user tasks, but leads to long event residence time. 2) Ellipsis adds additional template-matching overhead to Linux Auditd (Fig.1(a)) and uses a shared kernel buffer among multiple tasks. Thus, one task may cause queuing delays or event dropping for events generated by others.

IV. THREAT MODEL

Our threat model is similar to recent system auditing

studies[29], [1], [6], [4].

Scenario: We concentrate on real-time systems where tasks are periodic and must be completed before the deadline. The additional overhead of system auditing should not cause deadline violations, all while ensuring the integrity of events.

Attacker Capabilities&Objectives: We assume that an attacker exploits existing OS vulnerabilities to escalate privileges and tamper with the audit buffer before events are all securely committed. Their goal is to conceal their aforementioned malicious audit events, thereby erasing attack traces to evade detection by subsequent provenance analysis. Preventing the system from being compromised is not in our scope.

Event processing: We ignore events generated after the system compromise because they are not trustworthy anymore. Events are also usually stored and processed in the remote server for provenance analysis. Thus, attackers still need to tamper events even they choose to deactivate the IDS in the compromised system.

V. REAL-TIME AUDITING IMPLEMENTATION

This section introduces a real-time system auditing framework, RT-NODROP, that is predictable, secure, and efficient. In contrast to current frameworks that optimize performance and security in a best-effort way, real-time auditing instead avoids the pathological worst-case execution time and unbounded interference.

RT-NODROP is a variant of NODROP [5] designed to provide efficient consumer execution and guarantee event integrity. Similar to NODROP, RT-NODROP collects syscall events using the efficient `tracepoint` mechanism [39] in the kernel, temporarily saves events in a kernel buffer, and transmits events to the consumer with zero-copy by sharing the buffer with the consumer. The consumer is also implemented as a per-thread module in the monitored thread address space. To ensure system predictability and bounded event residence time, RT-NODROP implements a schedulability-aware consumer invocation mechanism. Concretely, RT-NODROP introduces the following optimizations to improve NODROP.

Periodic consumer invocation. NODROP does not invoke the consumer until the buffer is full, leading to burst execution and extremely long event residence time. In contrast, RT-NODROP periodically invokes the consumer to ensure schedulability. §VII-A will demonstrate adapting consumer execution to RTA, which calculates the consumer invocation period that satisfies the system’s schedulability. It is worth noting that the original NODROP, lacking such periodic execution, cannot achieve RTA. Controlled, periodic consumer execution yields several key advantages. 1) *Bounded consumer WCET.* The consumer invocation period limits the upper bound on the number of processed events, which primarily decides the consumer WCET. 2) *Bounded event residence time.* The event residence time is determined by the timing of consumer invocation and the amount of preemption from higher priority tasks. Given the consumer invocation period, §VII-B accurately derives the bound of event residence time via RTA. 3) *Bounded kernel buffer memory usage.* A periodic

consumer enables calculating the maximum space needed for the kernel buffer based on the invocation period and application syscall frequency. This prevents buffer overflow and application blocking.

Task-aware event residence time control. Some auditing frameworks, such as Sysdig and NODROP, introduce unpredictable event residence time, while OMNILOG is very aggressive in optimizing event residence time. OMNILOG achieves immediate transmission of all events at the cost of decreased performance. However, diverse tasks within a system may exhibit varying requirements concerning event residence time. We implement two optimizations to balance event residence time, schedulability, and system efficiency. 1) The consumer invocation period of each application is individually controlled based on its load and interference between applications. This enables each task to have a different and controlled event residence time. 2) System administrators can customize constraints and goals for event residence time. This flexibility allows administrators to prioritize, for instance, minimizing the event residence time for the highest-priority task or optimizing the average event residence time across all tasks. Furthermore, §VII-C demonstrates a consumer invocation period selection algorithm that calculates the period to optimize the specified event residence time objective while ensuring schedulability.

Decoupling consumer invocation from syscall execution.

Since NODROP only switches to the consumer at the end of the syscall and resumes the application execution until the consumer completes the event handling, the consumer incurs heavy latency to the intercepted syscall, significantly increasing the application tail latency. For example, when a service (such as Redis) handles user requests, the consumer invocation on the execution path of `socket` syscalls substantially increases the request tail latency. In contrast, RT-NODROP decouples the consumer invocation from syscall execution to reduce the latency of syscall-intensive applications, such as Redis. For each thread, RT-NODROP registers a timer during the consumer initialization and sets its expiration time as the calculated consumer invocation period. The timer binds a callback function to prepare the consumer execution context for that thread. When the timer is triggered, the kernel executes the callback function, which replaces the program counter with the consumer entry point, initializes other registers, and allocates a stack for the consumer. Therefore, when the monitored thread is scheduled, it will directly execute the consumer context. Similar to NODROP, when the consumer exits, it invokes a syscall to restore the original thread context and switch back to the monitored thread. Additionally, RT-NODROP refreshes the timer upon consumer exit. When the thread is destroyed, RT-NODROP deletes the corresponding timer of that thread.

Consumer pre-loading. The consumer initialization during its first run is expensive as it requires reading binary from the disk and parsing it to load into the application’s address space. NODROP loads the consumer on demand to mitigate the cost. However, this greatly increases the consumer WCET and is unsuitable for real-time systems. RT-NODROP adopts a

pre-loading approach, where consumers are loaded in advance at the time of each thread creation. In the subsequent thread execution, RT-NODROP only has the cost of switching to the consumer, without the cost of loading and initialization.

Discussion. We discuss several trade-offs in the RT-NODROP design. Compared to NODROP, we enhance schedulability by imposing bounded WCET and event residence time, albeit at the cost of more frequent consumer invocations, which leads to a slight performance degradation. In contrast to OMNILOG, we sacrifice a certain degree of event residence time to significantly improve system performance and schedulability. Additionally, we compromise the residence time of low-priority tasks to a certain extent to ensure the execution of high-priority tasks. This relies on administrators setting appropriate optimization objectives for event residence time. Compared to Ellipsis [25], [8], we elevate consumer priorities to guarantee security. The event reduction optimizations employed in Ellipsis are complementary to RT-NODROP. Determining the optimal priority for each consumer to further optimize the system is our future work. In case of malicious tasks that issue syscalls at an unbounded rate, RT-NODROP still can maintain system security. Unbounded syscall generation overflows the buffer before consumer invocation. We can detect the overflow and issue an alarm. Different systems can configure different respond strategies. 1) Fallback to the NODROP, which invokes the consumer immediately when the buffer is full. This ensures no events are lost, but more consumer invocation will cause additional interference to user tasks. 2) Drop events from the current task. Thanks to the per thread buffer design, this only discards events generated by malicious tasks and has no impact on other tasks. However, since the unbounded syscall execution of the malicious task already introduces interference, RT-NODROP cannot ensure the system's schedulability.

VI. SYSTEM MODEL AND ASSUMPTIONS

Auditing frameworks inevitably impose syscall tracing and consumer execution overhead to the system, which increases performance interference. Therefore, an auditing overhead-aware RTA is necessary to derive the bounds necessary for predictable system auditing. This section introduces the system model, which is similar to the one widely used in previous research [25], [40], [41], [42].

We consider a real-time system, consisting of N tasks $\tau = \{\tau_1, \tau_2, \dots, \tau_N\}$ running on M cores. Each task τ_i is characterized by a tuple (e_i, p_i, s_i) . e_i is WCET of task τ_i and p_i is the minimum arrival time between jobs of τ_i . Hard real-time systems require predictable task execution times to avoid deadline misses. Thus, every task must invoke only a finite number of syscalls, and we use s_i to represent the maximum number of syscalls invoked by τ_i . We also assume that the operating system ensures a bounded execution time of each syscall. This paper focuses on bounding the additional overhead and interference from auditing frameworks. The deadline and response time of a task are p_i and r_i , respectively. The utilization of a task is defined as $\frac{e_i}{p_i}$. We statically assign each task to a core and assume partitioned fixed-priority

scheduling, which is popular in real-time systems, such as AUTOSAR [43]. The set of tasks with higher (lower) priority than τ_i (on the same core as τ_i) is hp_i (lp_i).

Auditing frameworks typically introduce two categories of extra overhead: tracing overhead associated with each syscall and consumer execution overhead. We define Δ as the maximum overhead added to each syscall and ϵ as the WCET of a consumer. ϵ is composed of two sources of overhead: consumer invocation cost and syscall processing overhead. We let α denotes the constant invocation overhead of the consumer implementation, and β is the processing cost incurred for each syscall within the consumer. The invocation period of a consumer is q . R denotes the response time of the consumer task. Current auditing frameworks, such as Sysdig and OMNILOG, employ a single consumer. Therefore, we assume that the consumer has the highest priority, and its period is equal to that of the highest-priority real-time task (τ_1). This is consistent with the practical deployment of the current system auditing. Since RT-NODROP has a per-thread consumer execution model, we treat the consumer as a separate task. Concretely, for each task τ_i , there is a derived consumer task $\sigma_i = (\epsilon_i, q_i)$, whose priority is higher than τ_i but lower than any task in hp_i .

VII. RESPONSE-TIME AND RESIDENCE TIME ANALYSIS

This section conducts an auditing overhead-aware analysis to bound both the response time and event residence time of each task. Audsley et al. [44] established a bound on τ_i 's response time r_i .

$$r_i = e_i + b_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{r_i}{p_k} \right\rceil \times e_k \quad (1)$$

where b_i denotes the total blocking time incurred by τ_i . Similar to previous works [25], [40], [42], our subsequent analysis is based on this RTA method.

A. RT-NODROP Task Response Time Analysis

According to the previous research [25] on calculating r_i , we initially consider the extra tracing overhead imposed on each syscall, which slows down task execution, and then bound the consumer execution interference of RT-NODROP.

Increased task WCET. Given the maximum overhead imposed on each syscall (Δ) and that task τ_i executes s_i syscalls, the additional execution time for τ_i is $s_i \times \Delta$. Therefore, the expanded WCET of task τ_i is defined as E_i , where

$$E_i = e_i + s_i \times \Delta \quad (2)$$

Consumer execution interference. RT-NODROP introduces multiple consumer tasks that process audit events, and their overhead and interference cannot be ignored. To determine the total interference of RT-NODROP consumers, we first bound the WCET of each consumer execution and then calculate their interference on lower priority tasks.

The WCET of a consumer σ_i comprises two components: a constant consumer invocation overhead (α) including context switch and event transfer, and a processing overhead for each

syscall (β). Given the consumer invocation period (q_i), the maximum number of syscalls handled is $\lceil \frac{q_i}{p_i} \rceil \times s_i$. Thus, we have the consumer WCET as

$$\epsilon_i = \alpha + \left\lceil \frac{q_i}{p_i} \right\rceil \times s_i \times \beta \quad (3)$$

Similar to interference from higher priority tasks, τ_i will be preempted by all higher priority consumers. We use chp_i to represent the set of consumers with higher priority than τ_i , which consists of consumers derived from τ_i and tasks in hp_i . Therefore, the total consumer interference on τ_i is

$$I(\sigma_i) = \sum_{\sigma_k \in chp_i} \left\lceil \frac{r_i}{q_k} \right\rceil \times \epsilon_k \quad (4)$$

Finally, the complete response time r_i under RT-NODROP is

$$r_i = E_i + b_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{r_i}{p_k} \right\rceil \times E_k + I(\sigma_i) \quad (5)$$

B. RT-NODROP Event Residence Time Analysis

The event residence time of task τ_i , denoted as R_i , comprises two components: the waiting time T_i^w in the kernel buffer prior to consumer invocation and the processing time T_i^p inside the consumer [45], [46].

$$R_i = T_i^w + T_i^p \quad (6)$$

Obviously, the worst-case waiting time is bounded by the consumer invocation period, i.e., $T_i^w = q_i$.

Regarding the worst-case processing time T_i^p for an event, it can not be simply considered as the syscall processing overhead within the consumer or the WCET of the consumer, because higher priority tasks can preempt the consumer. Thus, a safe upper bound for T_i^p is the response time of the consumer task. Similar to the calculation of task response time, determining the consumer response time needs to combine the interference from both higher priority tasks and consumers. Applying the same analysis in §VII-A, we have

$$T_i^p = \epsilon_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{T_i^p}{p_k} \right\rceil \times E_k + \sum_{\sigma_k \in chp_i \setminus \sigma_i} \left\lceil \frac{T_i^p}{q_k} \right\rceil \times \epsilon_k \quad (7)$$

C. RT-NODROP Consumer Period Selection

In order to optimize the event residence time while ensuring the system schedulability, it is necessary to adjust the consumer invocation period (q_i) of each task. Finding the optimal period requires enumerating all combinations of all consumer periods and is NP-hard. Therefore, we use the particle swarm optimization (PSO) algorithm to solve this problem [47]. The core idea is to use N particles, regard each particle as a potential solution in the search space, and guide the position update through its own and the group's optimization experience during max_gen iterations, to collaboratively search for the global optimal solution. PSO exhibits good convergence speed and stability when dealing with high-dimensional complex optimization problems. During the iteration process, the update formula for particle speed and position is:

```

1 def PSO_OPTIMIZE_QK(ts, qmin, qmax, N, max_gen, w,
  c1, c2):
2     # Initialize particle swarm
3     ps = rand_int_vectors([ts], qmin, qmax, pop_size)
4     p_best = ps
5     g_best = arg minp ∈ ps event_residence_time(p)
6     v = zeros(shape=ps.shape)
7     for iter in range(max_gen):
8         for i in range(N):
9             p = ps[i]
10            if is_schedulable(p, ts):
11                ert = event_residence_time(p, ts)
12            else:
13                ert = float('inf')
14            # Update personal best
15            if ert < event_residence_time(p_best[i], ts):
16                p_best[i] = p
17            # Update global best
18            if ert < event_residence_time(g_best, ts):
19                g_best = p
20            r1, r2 = random(), random()
21            v[i] = w · v[i] + c1 · r1 · (p_best[i] - p) +
                c2 · r2 · (g_best - p)
22            ps[i] = clip(round(p + v[i]), qmin, qmax)
23        return g_best

```

Fig. 3: PSO-based Consumer Period Optimization

$$v(t+1) = w \cdot v(t) + c_1 r_1 \cdot (p_{best} - q(t)) + c_2 r_2 \cdot (g_{best} - q(t))$$

$$q(t+1) = \text{clip}(\text{round}(q(t) + v(t+1)), q_{min}, q_{max}) \quad (8)$$

where v is the particle velocity, q is the period vector, p_{best} and g_{best} denote the historical best particle position and the global best swarm position, respectively, and r_1, r_2 are two independent uniformly distributed random numbers.

We adapt PSO to the consumer period selection by using particles to represent possible period vectors $\mathbf{q} = \{q_1, q_2, \dots, q_n\}$. Fig. 3 shows the pseudo code of the algorithm, and the overall algorithm complexity is $O(max_gen \times N \times n)$, where n is the number of consumers. The algorithm parameters are set as follows: the inertia weight $w = 0.5$, and the learning factors $c_1 = c_2 = 1.5$. We first determine the possible range of period $[q_{min}, q_{max}]$. For τ_i , q_{min} is p_i and q_{max} satisfies $E_i + \lceil \frac{q_{max}}{p_i} \rceil \times s_i \times \beta = p_i$. During optimization, we use the user-specified event residence time as the fitness of the particle. If the current solution is not schedulable, its fitness is set to $+\infty$ and the particle is discarded.

Consumer period selection is NP-hard, thus we do not claim PSO as the best approach. However, as a meta heuristic algorithm, it can efficiently obtain approximate optimal solutions, is simple to use, and offers flexible parameter configuration to fine-tune the algorithm's accuracy and cost. Specifically, each particle position naturally represents a set of period multiple vectors corresponding to the task, achieving discrete search modeling for period selection, which converges faster than genetic algorithms. At the same time, PSO can jointly adjust the period of all tasks from a global perspective, enabling it to find a combination with shorter overall residence time while satisfying system schedulability.

D. Response Time Analysis for Existing Auditing Frameworks

Although existing auditing frameworks exhibit unpredictable behavior and their execution models do not strictly adhere to RTA constraints, this section simplifies their execution model to facilitate an optimistic analysis, aiming

to provide insights into design defects of current auditing frameworks when integrated into real-time systems. Despite the optimism in the analysis, the inherent priority inversion and the substantial system overheads significantly compromise the system schedulability. Specifically, we make the following simplifications. 1) We assume that the consumer operates periodically with the highest priority, thereby disregarding the potential consumer sleeping and reducing the number of accumulated events. Prior research assigns the consumer with the lowest priority [25]. However, this configuration leads to prolonged event residence time and a substantial number of events to be processed at once. Previous work confirms that the lowest-priority consumer still significantly degrades the schedulability [25]. Determining the optimal priority for consumers within existing auditing frameworks is out of the scope of this paper. 2) We assume that the consumer can handle events instantaneously, thus neglecting the overhead associated with consumer invocation (α) and decreasing the WCET of consumer execution. In practice, consumer invocation in existing frameworks involves a costly process of wake-up and context switching, 3) We assume that the consumer is capable of handling all events during its execution, thereby ensuring that no events are lost and that tasks do not await consumer availability. In reality, tasks often block to prevent generating events at a rate that would saturate the consumer [5]. This simplification consistently favor the baseline, because they neglect consumer invocation overhead, assume smaller WCET, and omit possible event dropping.

A key difference between existing auditing frameworks and RT-NODROP is that they employ a single consumer task (σ) to process audit events across all tasks, inevitably introducing priority inversion. Therefore, determining the WCET for a consumer depends on calculating the upper bound on the number of events the consumer handles.

Upper bound of syscall count. An established upper bound on the number of jobs issued by τ_i in any interval of length W is $\left\lceil \frac{W+r_i}{p_i} \right\rceil$ [48], which implies a bound on the maximum number of syscalls.

Lemma 1: Within a time window W , the maximum number of syscall task τ_i can invoke is

$$S_i(W) = \left\lceil \frac{W+r_i}{p_i} \right\rceil \times s_i \quad (9)$$

and the total number of syscalls across all tasks is

$$S(W) = \sum_{i=1}^N S_i(W) \quad (10)$$

Lemma 2: The maximum number of syscalls processed by the consumer is upper bounded by:

$$A^* = S(q+r_\sigma) = \sum_{i=1}^N \left\lceil \frac{q+r_\sigma+r_i}{p_i} \right\rceil \times s_i \quad (11)$$

Proof Sketch: An audit event will be transferred to the consumer at the latest when the next consumer task is released (consumer inoculation period q). r_σ , the consumer response

TABLE I: lmbench results in microsecond. *Smaller is better.*

	stat	read	write	open/close	null
Sysdig	1.85	1.10	1.00	3.69	0.86
Sysdig-dropfree	2.86	2.45	2.28	6.01	2.00
OMNILOG	3.80	3.28	3.19	7.80	3.06
Auditd	9.21	5.65	5.32	18.55	4.75
Ellipsis	7.37	4.52	4.26	14.84	3.80
NODROP	1.45	0.95	0.89	2.88	0.68
RT-NODROP	1.61	1.07	1.00	3.16	0.76

time, captures the maximum time required for the consumer to complete. Thus, at any point in time t , all audit events corresponding to syscalls before $t - q - r_\sigma$ must have been processed by the consumer. Consequently, the worst-case syscall count processed by the consumer is the maximum number of syscall generated by all tasks during a time interval of length $q + r_\sigma$.

Task response time. Since a single consumer task is employed, analyzing consumer interference is easier than RT-NODROP. By neglecting the consumer invocation overhead, the WCET of the consumer is $A^* \times \beta$. The complete response time of τ_i is,

$$r_i = E_i + b_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{r_i}{p_k} \right\rceil \times E_k + \left\lceil \frac{r_i}{q} \right\rceil \times A^* \times \beta \quad (12)$$

Ellipsis adopts the same analysis model as RT-NODROP while the only difference is the consumer priority. Ellipsis's consumer is lowest priority and will continue to run when there are no other high priority tasks running.

VIII. EXPERIMENTAL EVALUATION

We conduct experiments on a 6-core PC with E5-2520 v3 CPU @ 1.4GHz. The operating system is openEuler [49] based on the Linux 5.4 kernel, which enhances real-time system performance [34]. We compare RT-NODROP with Sysdig, OMNILOG, Auditd, Ellipsis, and NODROP. The default consumer period in RT-NODROP is 10ms.

A. Micro Benchmarks

Similar to the evaluations in other system auditing frameworks [5], [4], we use lmbench [50] to measure the syscall overhead caused by different auditing frameworks. We list the results in Table I.

Syscall performance. Our experiments show that the impact of RT-NODROP on syscalls is smaller than that of Sysdig and OMNILOG. The execution time of OMNILOG for all syscalls is up to 1.5 times higher than that of RT-NODROP. The primary reason lies in the inserted customized hypercall for event processing on every system call execution path in OMNILOG, which introduces an expensive VM-exit to switch to the hypervisor. The performance of RT-NODROP is up to 4 times higher than that of Ellipsis. That's because Ellipsis introduces a measurable but justifiable overhead primarily inherited from the underlying Linux Audit. The core cost stems from the mandatory execution of audit hooks, which involves rule checking, audit record construction with additional kmalloc operation, and operations on a shared kernel buffer protected

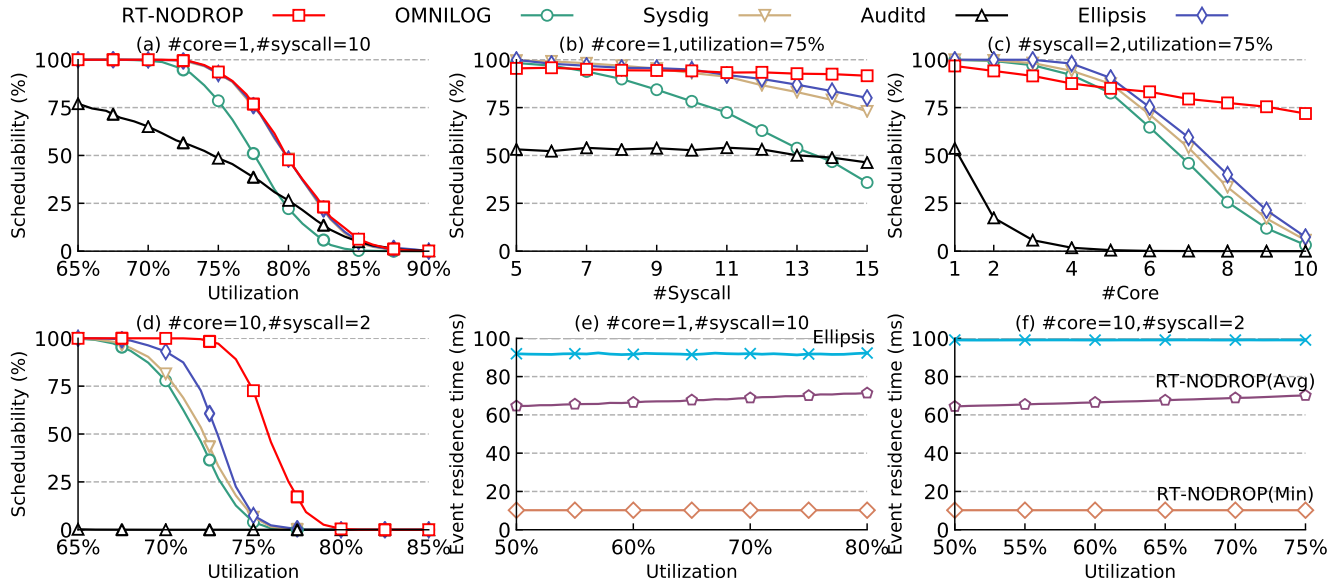


Fig. 4: Response time and event residence time analysis.

by a spinlock. This foundational auditing overhead constitutes the majority of the total latency increase. On top of this, Ellipsis adds its own logic for runtime template matching, which includes checking syscall sequences and arguments against a finite state automaton and managing temporary per-task buffers. Fortunately, when matches succeed, this cost is offset by reducing long syscall sequences to single records, actually making Ellipsis faster than standard Linux Audit. The syscall overhead of RT-NODROP is slightly higher than that of NODROP. For example, RT-NODROP only introduces 0.08 us more latency than NODROP for the NULL syscall. We conclude that this additional cost comes from the higher frequency of consumer invocation.

Event dropping. The Imbench results of Sysdig shows comparable performance with RT-NODROP, but it drops 67.5% of syscall events. We manually slow down Imbench to prevent event dropping in Sysdig, and report its results (Sysdig-dropfree). Table I shows that Sysdig further degrades the syscall performance when all events are processed. Other frameworks successfully capture all syscall events. While NODROP and RT-NODROP ensure no event dropping by design, OMNILOG achieves this due to its large syscall overhead that slows down the event generation.

Worst-case overhead. Based on Imbench experiment, we measure the worst-case cost required for RTA (§VII) and use these values for schedulability tests in §VIII-B. Concretely, for RT-NODROP, we have $\Delta = 0.31\mu s, \alpha = 200\mu s, \beta = 0.98\mu s$. For Sysdig, we have $\Delta = 0.31\mu s, \alpha = 0, \beta = 0.98\mu s$. For OMNILOG, we have $\Delta = 3.42\mu s, \alpha = 0, \beta = 0.98\mu s$. For Auditd, we have $\Delta = 7.04\mu s, \alpha = 0, \beta = 0.98\mu s$. For Ellipsis, we have $\Delta = 3.52\mu s, \alpha = 0, \beta = 0$.

B. Schedulability Test

Task set parameters. The schedulability test and analysis are from the SchedCAT toolkit [51]. The default RTA in

TABLE II: SUA improvement corresponding to Fig. 4

	Fig. 4(a)	Fig. 4(b)	Fig. 4(c)	Fig. 4(d)
Sysdig	0.67%	3.24%	24.15%	18.45%
OMNILOG	8.85%	26.24%	32.38%	20.54%
Auditd	32.30%	80.00%	1342.85%	1948.73%
Ellipsis	0.35%	1.71%	18.82%	13.67%

SchedCAT performs uniprocessor analysis for each individual core. We use the Emberson et al.’s generator [52] to generate a task set with 10 tasks per core. The period is uniformly chosen to be [10ms, 100ms]. We vary the per-core task set utilization across [0.6, 0.9]. The task execution cost is calculated based on the utilization and period. We study parameters of utilization, syscall count, and core count. We control the total syscall count by setting the number of syscalls per millisecond per task. For each configuration, we randomly generate 5000 samples to test schedulability.

Schedulability result. In a single core configuration, with the number of system calls fixed at 10, we observe that RT-NODROP performs well at lower utilization. For example, at 73% utilization, RT-NODROP’s schedulability remains around 100%. When utilization rises to 75%, the schedulability only slightly drops to 93%. After that, the schedulability begins dropping sharply. In contrast, OMNILOG performs much worse in the same configuration due to the large syscall overhead. At 73% utilization, its schedulability drops to 92%, and as utilization increases, the gap between OMNILOG and RT-NODROP is around 20% or even higher. Sysdig performs slightly worse than RT-NODROP due to omitted overhead and optimistic analysis. Ellipsis performs close to Sysdig, but it shows a little higher than Sysdig. Compared to these frameworks, Auditd drops earlier, and its performance is only about half that of RT-NODROP. At 65% utilization, its schedulability is 75%, and as utilization increases to 88%,

it drops relatively gently to 0%. However, as the number of syscall increases, from 5 to 15, Sysdig’s schedulability and Ellipsis’ schedulability declines much faster than RT-NODROP because it needs to process syscalls of all tasks, but RT-NODROP almost remains unchanged (Fig. 4 (b)). We can conclude that the priority inversion in Sysdig significantly hurts its schedulability. The schedulability of Auditd doesn’t drop sharply with the increase of syscall, but it always remains at a relatively low level. The Ellipsis’s performance advantage over Sysdig is not significant because the baseline (Auditd) is so slow that any efficient reduction mechanism built on top of it appears transformative. However, Ellipsis provides 88 times SUA improvement over Auditd which demonstrates that it is a real-time-capable auditing framework.

Since the schedulability of Sysdig and OMNILOG decreases substantially with the large core count, we have to perform multi-core analysis using a lower number of syscalls, as shown in Fig. 4 (c) and (d). At a utilization of 75%, as the number of cores increases, from 1 to 10, RT-NODROP’s schedulability only drops from 97% to 72%. Although under single core, When core count varies from 1 to 5, OMNILOG, Sysdig and Ellipsis have slightly higher schedulability than RT-NODROP due to small syscall count. When core count is 6, OMNILOG’s schedulability drops from 81% at 5 to 66%, Sysdig’s schedulability drops from 86% at 5 to 71%, and Ellipsis’s schedulability drops from 87% at 5 to 72%, but RT-NODROP’s schedulability only drops 1.7% between 5 and 6. When core count increases from 1 to 2, Auditd sharply drops from 50% to 20% and quickly drops to around 0% at 4. Comparing the results of single-core and multi-core configurations, RT-NODROP’s per-thread consumer model can effectively alleviate the audit overhead in multi-core environments, thus RT-NODROP is more scalable than Sysdig and OMNILOG. OMNILOG and Sysdig are limited by the single consumer that suffers from cross-core interference.

We also measure the improvement in the Schedulable Utilization Areas (SUAs). The SUA of an experiment is the area under its curve in the corresponding schedulability graph. Table II shows the improvement in SUA for RT-NODROP in each experiment in Fig. 4 over Sysdig and OMNILOG, respectively. On average, RT-NODROP outperforms Sysdig (OMNILOG) by 11.63% (21.98%) and outperforms Ellipsis (Auditd) by 8.64% (850.97%).

Event Residence Time Result. The results in Fig. 4 (e) and (f) indicate that the minimum event residence time remains relatively stable, consistently around 10ms. The average of event residence time in RT-NODROP exhibits a gradual increasing trend, starting from approximately 62ms at 50% utilization and incrementally rising to roughly 70ms after 70% utilization. This change is because more system calls increase the WCET of the consumer. Ellipsis has a longer residence time compared to RT-NODROP, which it remains from 90 to 100 ms, relatively stable. The result confirms that RT-NODROP maintains predictable and bounded minimal event residence time even under high system utilizations due to our overhead-aware RTA and effective period selection algorithm.

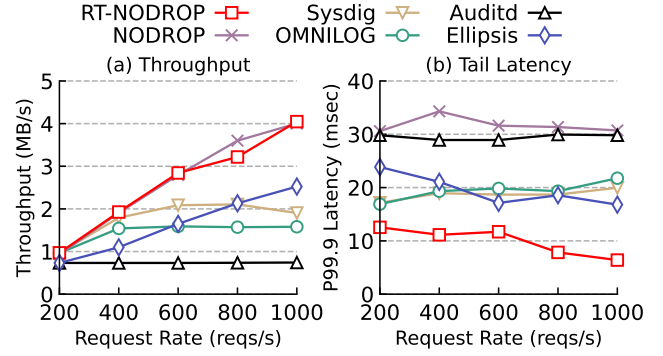


Fig. 5: Single-core Redis performance with controlled workload under different auditing frameworks.

C. Application Study: Redis

In this section, we illustrate how RT-NODROP provides lower tail latency for latency-sensitive applications while maintaining the same throughput with NODROP, of which we use Redis as a representative. Redis implements a key-value store serving *set/get* requests.

Experiment setup. Our experiment deploys the Redis with a single and six cores. We use *memtier_benchmark* to generate workloads which consist of various *set/get* requests ratios from 0% to 80%, data sizes from 32B to 1MB, and request rates from 200req/s to 1000req/s. For OMNILOG, the Redis server of both single core and six cores is deployed on a virtual machine, which is assigned six vCPUs and 10GB DRAM. We repeat each experiment ten times and report the average metrics, including request throughput and the P99.9 tail latency. The Ellipsis framework was configured following its standard two-phase methodology. During the offline learning phase, Redis are executed under standard Auditd in a controlled environment. The generated audit logs were processed to automatically extract frequent syscall sequences and compile them into kernel-level templates. For evaluation, these templates were loaded via the extended *auditctl* utility, with Ellipsis-HP enabled for maximal log reduction. The *kaudit_buffer* was set to 50,000 records to prevent overflow.

Controlled workloads. We first apply admission control to Redis, which limits the rate of requests and simulates periodic execution of Redis. The Redis throughput and tail latency of Redis under different controlled workloads are shown in Fig. 5. In this experiment, the *set* ratio is 20% and the data size is 32B. Since all six auditing frameworks bring less overhead to the Redis server, as fewer events are generated when the workload is low, they have a similar throughput. When the workload becomes high, the event is much more frequent and brings more overhead to the Redis server as Sysdig preempts it, leading to larger tail latency. RT-NODROP remains the smaller P99.9 tail latency compared to NODROP despite the request rate, as the periodic consumer invocation limits the upper bound on the number of processed events and the CPU cycles spent to process them without interrupting system calls.

We next relax the admission control and allow the client to send request at the maximum rate in a closed loop, which is

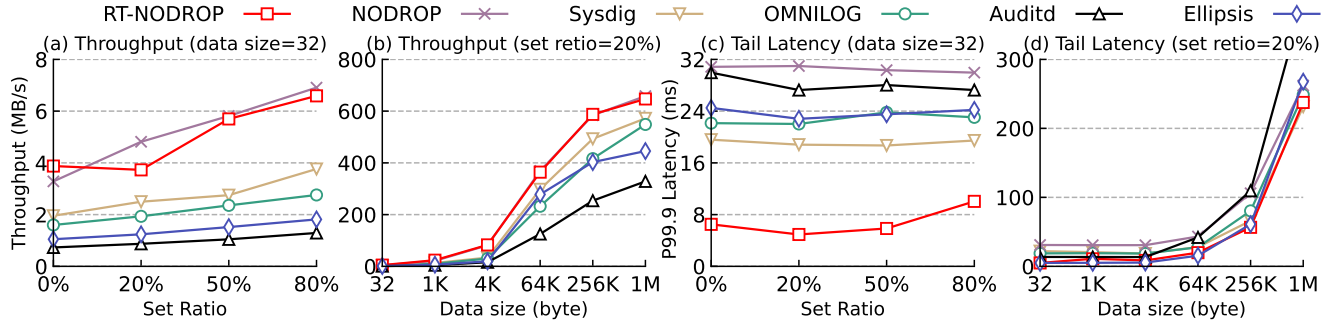


Fig. 6: Redis performance on single core under different auditing frameworks.

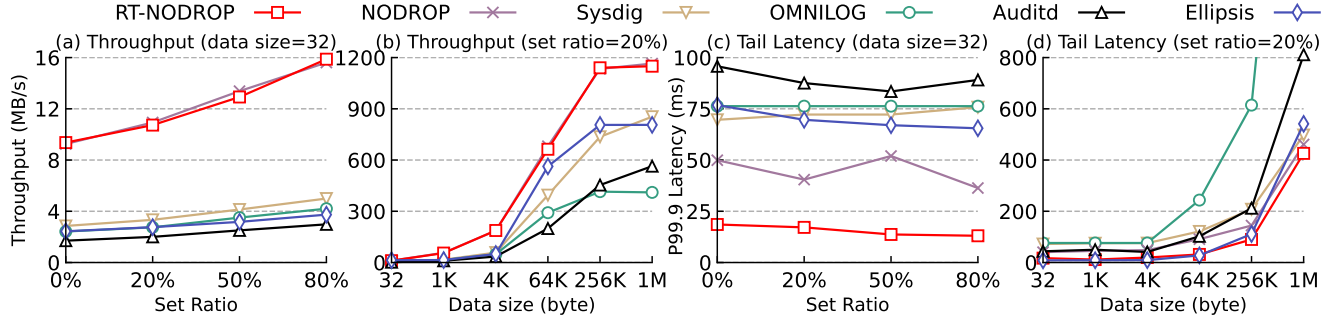


Fig. 7: Redis performance on multi cores under different auditing frameworks.

more practical. Furthermore, this enables the consumer to use slack time to improve overall system efficiency.

Throughput. As shown in Fig. 6 and Fig. 7, the throughput of RT-NODROP is essentially close to that of NODROP, as the periodic consumer invocation does not penalize the Redis server performance. Moreover, RT-NODROP significantly outperforms Sysdig, OMNILOG, Auditd and Ellipsis under both Redis server configurations (one core and six cores). Our experiments show that RT-NODROP achieves up to 75.1% and 138.86% higher throughput compared with Sysdig and OMNILOG, respectively. Our experiments also show that Auditd and Ellipsis are even worse than Sysdig and OMNILOG. Compared to Sysdig, the major performance benefits of RT-NODROP come from eliminating process scheduling overhead thanks to the lightweight threadlet execution. OMNILOG incurs substantial virtualization overhead, thus always achieves the lowest throughput. In addition to the throughput improvement, RT-NODROP also improves memory utilization due to its periodic execution. Current frameworks tend to use large buffers to mitigate event dropping, but RT-NODROP is able to accurately determine its buffer size based on its WCET bounds. As a result, in the Redis experiment, RT-NODROP saves dozens of times more memory than other frameworks.

Tail latency. Tail latency is crucial for latency-sensitive applications because it represents the quality of service and service level agreement of customers.

For the single-core scenario, as shown in Fig. 6(c) and (d), the tail latency remains insensitive to variations in the set request ratio, while it tends to increase as the data size grows larger. RT-NODROP always has the lowest P99.9 latency

among all auditing frameworks under the fixed set ratio and data size. Our experiments show that RT-NODROP achieves an average of 2.19, 3.07, 6.82 and 5.02 times lower P99.9 tail latency than Sysdig, OMNILOG, Auditd and Ellipsis. With a single core, the consumer is scheduled for event processing and preempts the Redis server, leading to higher tail latency. Moreover, RT-NODROP has lower tail latency compared to NODROP thanks to the decoupled and periodic consumer invocation. NODROP spends more CPU cycles to process a larger number of events for each consumer invocation when its large buffer (8MB) is full, bringing a long stall time to the interrupted system call. RT-NODROP invokes the consumer to process events periodically instead of waiting until the buffer is full, which will spend fewer CPU cycles for event processing without interrupting system calls because of the decoupling of consumer invocation from the syscall execution (§V). For the multi-core scenario, as shown in Fig. 7(c) and (d), RT-NODROP still demonstrates the best tail latency among other auditing frameworks.

Fig. 8 shows the distribution of the set/get latency. NODROP and RT-NODROP shows a similar distribution on both single-core and multi-core, but NODROP has a much longer tail. Sysdig and OMNILOG has worse overall distribution and longer tail latency than NODROP, while Auditd and Ellipsis are even worse, validating the predictability guarantees in RT-NODROP. Auditd shows the worst results of both set and get. The latency of Ellipsis is more than four times that of RT-NODROP.

Event residence time. The duration of events that reside in the insecure kernel buffer reflects the resistance to the race

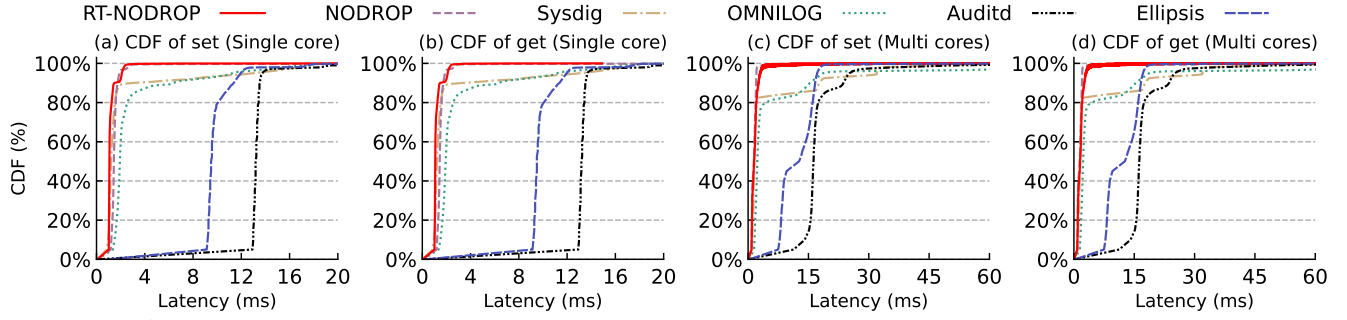


Fig. 8: The cumulative distribution function of Redis set/get latency under different auditing frameworks.

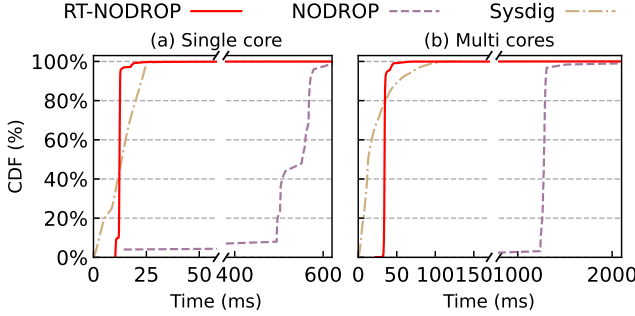


Fig. 9: The cumulative distribution function of event residence time under Sysdig, NODROP, and RT-NODROP.

condition attack. We empirically evaluate how long an event resides in the kernel buffer for the four auditing frameworks when the Redis server is running on both single core and multi-core scenarios. We omit the results of Ellipsis because most of events are not entering the kernel buffer.

To display the event residence time more intuitively, we measured the cumulative distribution function (CDF). The results are shown in Fig 9. RT-NODROP always achieves the lowest P99 event residence time. The CDF result shows that, when the Redis server is running on a single core, 99% of the RT-NODROP’s events reside for less than 20ms in the kernel buffer, which is much smaller than that of Sysdig and NODROP. 97% of NODROP’s events reside in the buffer for more than 500ms. We conclude that it adopts a huge buffer size and needs to wait a long time for the buffer to reach its capacity and switch to the consumer. It is worth noting that, when an application generates the system call events at a lower rate, the buffer will be extremely hard to fill, and the event residence time may be even higher.

Memory overhead. RT-NODROP’s memory consumption is bounded. The buffer size is calculated based on the syscall rate, audit event size, and consumer invocation period. In contrast, existing solutions set their buffer sizes empirically without guaranteed security. In multi-core Redis experiment, Sysdig, OMNILOG and NODROP utilize default 8MB buffer, resulting in 48MB memory occupation in total. Linux Audit and Ellipsis occupies 5MB. RT-NODROP only necessitates 256KB buffer per thread and a total memory usage of 1.5MB.

IX. RELATED WORK

Auditing on IoT devices. Approaches are focusing on applying auditing to IoT devices. Breitenbacher

proposed HADES-IoT, which intercepts `execve` system calls for analysis. Yu proposed ARTISAN [53], which records system state information in the spatial dimension by periodically taking file system snapshots and only logging the changes. Bansal et al. proposed Ellipsis [8], [25], a Linux Auditd-based auditing framework especially for real-time systems. It analyzes audit logs to template normal system call sequences in real-time systems, then compares these templates with actual event streams during execution, retaining only anomalous log sequences to minimize interference with real-time applications. This template matching method can also be adapted to RT-NODROP. RT-NODROP allows to apply conventional RTA to conduct a formal theoretical analysis. Moreover, this analysis necessitates a joint consideration of both task response time (schedulability) and event residence time (security). None of the existing frameworks, including Ellipsis, undertake a combined analysis of these two factors.

Investigations for Attack. There are many downstream attack investigation works related to system auditing that dig deeper into audit logs by translating event logs into provenance graphs [54], [55], [56], [57], [58]. Some methods are working for automatically scanning security risks [59], [60], [61], [62], [63], [64], [65]. Some methods are working for answering process queries of provenance graphs quickly and accurately [66], [67], [68], [69], [70], [71]. RT-NODROP benefits approaches mentioned above by providing a more reliable and usable system log collection tool for real-time system environments.

X. CONCLUSION

This paper presents RT-NODROP, a secure and predictable system auditing for real-time systems by introducing a threadlet-based execution model that provides bounds on WCET and event residence time, while preventing event dropping. We integrate the auditing overhead into RTA and provide a period selection algorithm to jointly calculate bounds for event processing interference and event residence time. Evaluations demonstrate that RT-NODROP improves schedulability by 80.11%, 117.9% and 51.05% over existing solutions like Sysdig, OMNILOG and Ellipsis. In experiments, Redis also achieves up to 75.1%, 138.86% and 324.6% higher throughput and 2.19, 3.07 and 5.02 times lower 99.9th percentile tail latency than Sysdig, OMNILOG and Ellipsis, respectively.

XI. ACKNOWLEDGEMENT

We sincerely thank all our anonymous reviewers for their valuable comments. This work was partly supported by the National Key RD of China (2022YFB4501802), the National Natural Science Foundation of China (62502088, 62172009, 62141208), Natural Science Foundation of Jiangsu Province (BK20251283) and Huawei Research Fund. Yuxin Ren and Ding Li are both the corresponding author.

REFERENCES

- [1] A. Ahmad, S. Lee, and M. Peinado, "Hardlog: Practical tamper-proof system auditing using a novel audit device," in *2022 IEEE Symposium on Security and Privacy (SP'22)*, 2022.
- [2] Redhat, "The linux audit framework," 2017, <https://github.com/linux-audit/>.
- [3] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eysers, M. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in *Proceedings of the 2017 Symposium on Cloud Computing (SoCC'17)*, 2017.
- [4] P. Jiang, H. Jiang, R. Huang, H. Lei, Z. Zhong, S. Zhang, Y. Ren, N. Jia, X. Hu, Y. Guo, X. Chen, and D. Li, "Dpuaudit: Dpu-assisted pull-based architecture for near-zero cost system auditing," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA'25)*, 2025.
- [5] P. Jiang, R. Huang, D. Li, Y. Guo, X. Chen, J. Luan, Y. Ren, and X. Hu, "Auditing frameworks need resource isolation: A systematic study on the super producer threat to system auditing and its mitigation," in *32nd USENIX Security Symposium (USENIX Security'23)*, 2023.
- [6] V. Gandhi, S. Banerjee, A. Agrawal, A. Ahmad, S. Lee, and M. Peinado, "Rethinking system audit architectures for high event coverage and synchronous log availability," in *32nd USENIX Security Symposium (USENIX Security'23)*, 2023.
- [7] V. T. Hoang, C. Wu, and X. Yuan, "Faster yet safer: Logging system via Fixed-Key blockcipher," in *31st USENIX Security Symposium (USENIX Security'22)*, 2022.
- [8] A. Bansal, A. Kandikuppa, C.-Y. Chen, M. Hasan, A. Bates, and S. Mohan, "Towards efficient auditing for real-time systems," in *Computer Security – ESORICS 2022: 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26–30, 2022, Proceedings, Part III*, 2022.
- [9] Sysdig. (2023) Sysdig: Secure, monitor, and troubleshoot your cloud. Container security and monitoring platform. [Online]. Available: <https://sysdig.com/>
- [10] W. Ul Hassan, M. Nouredine, P. Datta, and A. Bates, "Omegalog: High-fidelity attack investigation via transparent multi-layer log analysis," in *27th Annual Network and Distributed System Security Symposium (NDSS'20)*, 2020.
- [11] W. Ul Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, and A. Bates, "Nodoze: Combatting threat alert fatigue with automated provenance triage," in *26th Annual Network and Distributed System Security Symposium (NDSS'19)*, 2019.
- [12] Q. Wang, W. U. Hassan, D. Li, K. Jee, X. Yu, K. Zou, J. Rhee, Z. Chen, W. Cheng, C. A. Gunter, and H. Chen, "You are what you do: Hunting stealthy malware via data provenance analysis," in *27th Annual Network and Distributed System Security Symposium (NDSS'20)*, 2020.
- [13] A. Bates, D. Tian, K. R. B. Butler, and T. Moyer, "Trustworthy whole-system provenance for the linux kernel," in *Proceedings of the 24th USENIX Conference on Security Symposium (USENIX Security'15)*, 2015.
- [14] Y. Liu, M. Zhang, D. Li, K. Jee, Z. Li, Z. Wu, J. Rhee, and P. Mittal, "Towards a timely causality analysis for enterprise security," in *25th Annual Network and Distributed System Security Symposium (NDSS'18)*, 2018.
- [15] M. N. Hossain, S. M. Milajerdi, J. Wang, B. Eshete, R. Gjomemo, R. Sekar, S. D. Stoller, and V. N. Venkatakrishnan, "Sleuth: real-time attack scenario reconstruction from cots audit data," in *Proceedings of the 26th USENIX Conference on Security Symposium (USENIX Security'17)*, 2017.
- [16] M. A. Inam, W. U. Hassan, A. Ahad, A. Bates, R. Tahir, T. Xu, and F. Zaffar, "Forensic analysis of configuration-based attacks," in *29th Annual Network and Distributed System Security Symposium (NDSS'22)*, 2022.
- [17] S. Ma, X. Zhang, and D. Xu, "Protracer: Towards practical provenance tracing by alternating between logging and tainting," in *Proceedings 2016 Network and Distributed System Security Symposium (NDSS'16)*, 2016.
- [18] Sysdig, "Sysdig expands presence in india with launch of the first real-time cloud security saas platform," 2024. [Online]. Available: <https://sysdig.com/press-releases/sysdig-launches-saas-platform-in-india/>
- [19] F. Dong, S. Li, P. Jiang, D. Li, H. Wang, L. Huang, X. Xiao, J. Chen, X. Luo, Y. Guo, and X. Chen, "Are we there yet? an industrial viewpoint on provenance-based endpoint detection and response tools," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS'23)*, 2023.
- [20] "Sysdig 2023 cloud-native security and usage report," <https://sysdig.com/2023-cloud-native-security-and-usage-report/>.
- [21] M. Hasan, A. Kashinath, C.-Y. Chen, and S. Mohan, "Sok: Security in real-time systems," *ACM Comput. Surv.*, vol. 56, no. 9, Apr. 2024.
- [22] C.-Y. Chen, S. Mohan, R. Pellizzoni, R. B. Bobba, and N. Kiyavash, "A novel side-channel in real-time schedulers," in *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'19)*, 2019.
- [23] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Experimental security analysis of a modern automobile," in *2010 IEEE Symposium on Security and Privacy (SP'10)*, 2010.
- [24] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesner, and T. Kohno, "Comprehensive experimental analyses of automotive attack surfaces," in *Proceedings of the 20th USENIX Conference on Security*, 2011.
- [25] A. Bansal, A. Kandikuppa, M. Hasan, C.-Y. Chen, A. Bates, and S. Mohan, "System auditing for real-time systems," *ACM Trans. Priv. Secur.*, vol. 26, no. 4, Nov. 2023.
- [26] 5G is all about Speed & Latency! Huawei Enterprise Support. [Online]. Available: <https://forum.huawei.com/enterprise/en/5g-is-all-about-speed-latency/thread/667260360132018176-667213872962088960>
- [27] 5G vision white paper. SAMSUNG Research. [Online]. Available: <https://images.samsung.com/is/content/samsung/p5/global/business/networks/insights/white-paper/5g-vision/global-networks-insight-samsung-5g-vision-2.pdf>
- [28] B. S. Khan, S. Jangsher, A. Ahmed, and A. Al-Dweik, "Ullc and embb in 5g industrial iot: A survey," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1134–1163, 2022.
- [29] R. Paccagnella, K. Liao, D. Tian, and A. Bates, "Logging to the danger zone: Race condition attacks and defenses on system audit frameworks," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS'20)*, 2020.
- [30] P. Kogge, "Of piglets and threadlets: Architectures for self-contained, mobile, memory programming," in *Innovative Architecture for Future Generation High-Performance Processors and Systems (IWIA'04)*, 2004.
- [31] S. Park, S. Lee, W. Xu, H. Moon, and T. Kim, "libmpk: Software abstraction for intel memory protection keys (Intel MPK)," in *2019 USENIX Annual Technical Conference (USENIX ATC'19)*, 2019.
- [32] A. Vahldiek-Oberwagner, E. Elnikety, N. O. Duarte, M. Sammler, P. Druschel, and D. Garg, "ERIM: Secure, efficient in-process isolation with protection keys (MPK)," in *28th USENIX Security Symposium (USENIX Security'19)*, 2019.
- [33] W. Shao, X. Han, E. Stella, L. Dierksheide, P. K. Gadepalli, and G. Parmer, "Janus: Os support for a secure, fast control-plane," in *2025 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'25)*, 2025.
- [34] Z. Deng, Z. Zhang, D. Li, Y. Guo, Y. Ye, Y. Ren, N. Jia, and X. Hu, "Interference-free operating system: A 6 years' experience in mitigating cross-core interference in linux," in *2024 IEEE Real-Time Systems Symposium (RTSS'24)*, 2024.
- [35] J. H. Ryoo, N. Gulur, S. Song, and L. K. John, "Rethinking tlb designs in virtualized environments: A very large part-of-memory tlb," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA'17)*, 2017.
- [36] R. Pan, G. Peach, Y. Ren, and G. Parmer, "Predictable virtualization on memory protection unit-based microcontrollers," in *2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'18)*, 2018.
- [37] H. Huang, J. Lai, J. Rao, H. Lu, W. Hou, H. Su, Q. Xu, J. Zhong, J. Zeng, X. Wang, Z. He, W. Han, J. Liu, T. Ma, and S. Wu, "Pvm: Efficient shadow paging for deploying secure containers in cloud-native

- environment,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP’23)*, 2023.
- [38] J. Zhang, W. Jia, S. Chai, P. Liu, J. Kim, and T. Xu, “Direct memory translation for virtualized clouds,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS’24)*, 2024.
- [39] M. Desnoyers, “Using the linux kernel tracepoints <https://www.kernel.org/doc/html/latest/trace/tracepoints.html>.”
- [40] Y. Ren, G. Liu, G. Parmer, and B. Brandenburg, “Scalable memory reclamation for multi-core, real-time systems,” in *2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS’18)*, 2018.
- [41] M. Hasan, S. Mohan, R. B. Bobba, and R. Pellizzoni, “Beyond just safety: Delay-aware security monitoring for real-time control systems,” *ACM Trans. Cyber-Phys. Syst.*, vol. 6, no. 3, Sep. 2022.
- [42] —, “Exploring opportunistic execution for integrating security into legacy hard real-time systems,” in *2016 IEEE Real-Time Systems Symposium (RTSS’16)*, 2016.
- [43] “AUTOSAR os specification: <http://www.autosar.org/>.”
- [44] N. Audsley, A. Burns, M. Richardson, K. Tindell, and A. Wellings, “Applying new scheduling theory to static priority pre-emptive scheduling,” *Software Engineering Journal*, vol. 8, pp. 284–292, 1993.
- [45] C. Hamann, “On the quantitative specification of jitter constrained periodic streams,” in *Proceedings of the 5th International Workshop on Modeling, Analysis, and Simulation of Computer and Telecommunications Systems*, ser. MASCOTS ’97. USA: IEEE Computer Society, 1997, p. 171.
- [46] J. W. S. Liu, in *Real-Time Systems*, 2000.
- [47] R. Eberhart and J. Kennedy, “A new optimizer using particle swarm theory,” in *Proceedings of the Sixth International Symposium on Micro Machine and Human Science (MHS’95)*. Nagoya, Japan: IEEE, 1995, pp. 39–43.
- [48] B. B. Brandenburg, “Scheduling and locking in multiprocessor real-time operating systems,” Ph.D. dissertation, University of North Carolina at Chapel Hill, 2011.
- [49] openEuler, “<https://www.openeuler.org/>.”
- [50] L. W. McVoy, C. Staelin *et al.*, “Lmbench: Portable tools for performance analysis,” in *USENIX annual technical conference (ATC’96)*, 1996.
- [51] S. schedulability test collection and toolkit, “<http://www.mpsys.org/bb/projects/schedcat>.”
- [52] P. Emberson, R. Stafford, and R. I. Davis, “Techniques for the synthesis of multiprocessor tasksets,” in *proceedings 1st International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS 2010)*, 2010, pp. 6–11.
- [53] L. Yu, Y. Ye, Z. Zhang, and X. Zhang, “Cost-effective attack forensics by recording and correlating file system changes,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 1705–1722. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/you-le>
- [54] A. Gehani and D. Tariq, “Spade: Support for provenance auditing in distributed environments,” in *Middleware 2012*, 2012, pp. 101–120.
- [55] S. T. King and P. M. Chen, “Backtracking intrusions,” in *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles (SOSP’03)*, 2003.
- [56] D. J. Pohly, S. McLaughlin, P. McDaniel, and K. Butler, “Hi-fi: collecting high-fidelity whole-system provenance,” in *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC’12)*, 2012.
- [57] P. Jiang, J. Xiao, D. Li, H. Yu, Y. Bai, Y. Guo, and X. Chen, “Detecting malicious websites from the perspective of system provenance analysis,” *IEEE Transactions on Dependable and Secure Computing (TDSC’23)*, 2023.
- [58] S. Li, F. Dong, X. Xiao, H. Wang, F. Shao, J. Chen, Y. Guo, X. Chen, and D. Li, “Nodlink: An online system for fine-grained apt attack detection and investigation,” in *NDSS’24*, 2024.
- [59] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, “Unicorn: Runtime provenance-based detector for advanced persistent threats,” in *Network and Distributed System Security Symposium (NDSS’20)*, 2020.
- [60] Z. Gu, K. Pei, Q. Wang, L. Si, X. Zhang, and D. Xu, “Leaps: Detecting camouflaged attacks with statistical learning guided by program analysis,” in *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 2015.
- [61] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, and A. Bates, “Nodoze: Combatting threat alert fatigue with automated provenance triage,” in *Network and Distributed Systems Security Symposium (NDSS’19)*, 2019.
- [62] M. N. Hossain, S. M. Milajerdi, J. Wang, B. Eshete, R. Gjomemo, R. Sekar, S. Stoller, and V. Venkatakrishnan, “Sleuth: Real-time attack scenario reconstruction from cots audit data,” in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.
- [63] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, “Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS’19)*, 2019.
- [64] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, “Holmes: real-time apt detection through correlation of suspicious information flows,” in *2019 IEEE Symposium on Security and Privacy (SP’19)*, 2019.
- [65] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, “Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS’19)*, 2019.
- [66] P. Gao, X. Xiao, D. Li, Z. Li, K. Jee, Z. Wu, C. H. Kim, S. R. Kulkarni, and P. Mittal, “Saql: A stream-based query system for real-time abnormal system behavior detection,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.
- [67] P. Gao, X. Xiao, Z. Li, F. Xu, S. R. Kulkarni, and P. Mittal, “Aiql: Enabling efficient attack investigation from system monitoring data,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018.
- [68] Y. Ji, S. Lee, E. Downing, W. Wang, M. Fazzini, T. Kim, A. Orso, and W. Lee, “Rain: Refinable attack investigation with on-demand inter-process information flow tracking,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS’17)*, 2017.
- [69] Y. Ji, S. Lee, M. Fazzini, J. Allen, E. Downing, T. Kim, A. Orso, and W. Lee, “Enabling refinable cross-host attack investigation with efficient data flow tagging and tracking,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.
- [70] Y. Liu, M. Zhang, D. Li, K. Jee, Z. Li, Z. Wu, J. Rhee, and P. Mittal, “Towards a timely causality analysis for enterprise security,” in *NDSS’18*, 2018.
- [71] T. Pasquier, X. Han, T. Moyer, A. Bates, O. Hermant, D. Eysers, J. Bacon, and M. Seltzer, “Runtime analysis of whole-system provenance,” in *Conference on Computer and Communications Security (CCS’18)*, 2018.